

Problème d'activation de capteurs pour surveillance de zones

Techniques d'optimisation · Karine Deschinkel
Groupe · Taskin Semih – Aslan Semih

Langage : Python 3.x
Solveur : GLPK 4.65 (glpsol)
Heuristique : Aléatoire +
Nettoyage

PARTIE 3 — CONSTRUCTION DES CONFIGURATIONS ÉLÉMENTAIRES & PROGRAMME LINÉAIRE

HEURISTIQUE DE GÉNÉRATION (10 000 ITÉRATIONS)

- 1 **Construction aléatoire** : mélanger aléatoirement les N capteurs, les ajouter un par un jusqu'à couvrir l'intégralité des M zones (stratégie gloutonne aléatoire).
- 2 **Phase de nettoyage (minimalité)** : parcourir chaque capteur de la configuration ; si sa suppression ne détruit pas la couverture totale, le retirer définitivement. Répéter jusqu'à stabilité → configuration *minimale*.
- 3 **Stockage** : ajouter la configuration nettoyée (représentée comme un frozenset) à l'ensemble global. Les doublons sont automatiquement ignorés.
- 4 **Résultat** : K configurations élémentaires uniques, toutes minimales (aucun capteur superflu).

Propriété clé : la phase de nettoyage garantit que chaque configuration est *élémentaire* — retirer n'importe quel capteur laisse au moins une zone non couverte. Cela réduit le nombre de variables du LP et améliore la qualité de la solution.

PROGRAMME LINÉAIRE (FORMAT CPLEX LP)

```
\ K configs C_1..C_k, N capteurs Maximize obj:  
t_1 + t_2 + ... + t_k Subject To \ Pour chaque  
capteur i : c_i: \sum_{k: i \in C_k} t_k \leq batterie_i  
Bounds 0 \le t_k pour tout k = 1..K
```

INTERPRÉTATION

- **Objectif** : maximiser la somme des temps d'activation = durée de vie totale du réseau.
- ⚡ **Contrainte énergie** : un capteur ne peut être actif plus longtemps que sa batterie, toutes configurations confondues.
- ✓ **Résolution** : glpsol --lp modele.lp -o solution.txt via la méthode du simplexe (variables continues).

VALIDATION — EXEMPLE DU SUJET (N=4, M=3)

DONNÉES

Capteur	Zones	Batterie
s ₁	Z ₁ , Z ₂	6
s ₂	Z ₂ , Z ₃	3
s ₃	Z ₃	2
s ₄	Z ₁ , Z ₃	6

SOLUTION OBTENUE (4 CONFIGS ÉLÉMENTAIRES)

Config	Capteurs	Durée t*
u ₁	{s ₂ , s ₄ }	2.5
u ₂	{s ₁ , s ₂ }	0.5
u ₃	{s ₁ , s ₃ }	2.0
u ₄	{s ₁ , s ₄ }	3.5

✓ **Durée de vie = 8.5 unités de temps (conforme à la solution théorique du sujet)**

Expérimentation & Analyse des résultats

Résultats sur les instances de l'annexe · Influence du nombre de configurations

10 000 itérations
Heuristique aléatoire
Simplexe GLPK

PARTIE 4 — EXPÉRIMENTATION SUR LES INSTANCES DE L'ANNEXE

Instance	N (capteurs)	M (zones)	Configs générées	Durée de vie optimale	Statut	Temps CPU
fichier-exemple.txt	4	3	4	8.5000	OPTIMAL	0.2 s
instance_exemple.txt	5	4	4	22.0000	OPTIMAL	0.2 s
moyen_test_3.txt	10	10	19	395.0000	OPTIMAL	0.2 s
moyen_test_2.txt	20	10	237	104.0000	OPTIMAL	0.2 s
gros_test_1.txt	100	200	6 722	2 933.6195	OPTIMAL	1.7 s
maxi_test_1.txt	1 000	500	9 880	19 550.2789	OPTIMAL	11.7 s

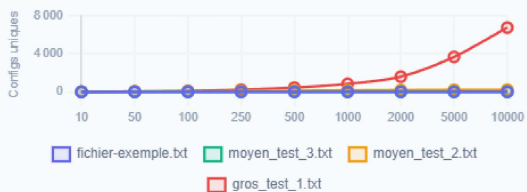
Toutes les instances sont résolues à l'optimal, y compris la plus grande (N=1 000, M=500, ~9 880 configurations). La méthode du simplexe sur des LP continus reste très efficace même à grande échelle.

PARTIE 5 — INFLUENCE DU NOMBRE ET DU TYPE DE CONFIGURATIONS SUR LA DURÉE DE VIE

DURÉE DE VIE VS. NOMBRE D'ITÉRATIONS (MOYENNE DE 3 RÉPÉTITIONS)



CONFIGURATIONS UNIQUES GÉNÉRÉES



DONNÉES — INSTANCE GROS_TEST_1.TXT (N=100, M=200)

Itérations	Configs	Durée moy.	Durée max
10	10	399.3	519.0
50	49	1 378.7	1 498.0
100	98	1 850.6	1 928.4
250	234	2 287.1	2 322.6
500	449	2 519.2	2 541.4
1 000	848	2 658.2	2 665.0
2 000	1 595	2 780.5	2 788.9
5 000	3 673	2 877.8	2 881.3
10 000	6 752	2 933.0	2 938.7

ANALYSE & CONCLUSIONS

- **Effet de saturation** : le gain marginal diminue avec le nombre d'itérations. Entre 10 et 100 iter : +363% de durée de vie. Entre 1 000 et 10 000 : seulement +10%.
- **Petites instances (N ≤ 20)** : l'espace de configurations est petit, l'optimum est atteint dès ~50–100 itérations. Davantage d'itérations n'apporte aucun gain.
- **Grandes instances (N ≥ 100)** : l'espace est immense, les nouvelles configurations apportent un gain régulier. La durée de vie passe de 399 (10 iter) à 2 933 (10 000 iter), soit ×7.3.
- **Robustesse** : dès ~100 itérations sur les instances moyennes, les résultats sont stables entre répétitions (min ≈ max). L'heuristique est fiable.
- **Type des configurations** : la phase de nettoyage (minimalité) est cruciale — elle évite les configs redondantes et réduit la taille du LP, accélérant la résolution sans perte de qualité.